

Local-Expansion Model

November 4, 2015

$ast ::= var \mid \mathbf{APP}(ast, ast, \dots) \mid val$
 $var ::= \mathbf{VAR}(name)$
 $val ::= \mathbf{FUN}(var, ast) \mid atom \mid \mathbf{LIST}(val, \dots) \mid stx$
 $stx ::= \mathbf{STX}(atom, ctx) \mid \mathbf{STX}(\mathbf{LIST}(stx, \dots), ctx)$
 $id ::= \mathbf{STX}(sym, ctx)$
 $ctx ::= \text{a mapping from } ph \text{ to } \overline{scp}$
 $\overline{scp} ::= \{scp, \dots\}$
 $atom ::= sym \mid prim \mid \dots$
 $sym ::= 'name$
 $prim ::= \mathbf{stx-e} \mid \mathbf{mk-stx} \mid \dots$
 $\xi ::= \text{a mapping from } name \text{ to } transform$
 $transform ::= \text{lambda} \mid \text{let-syntax} \mid \text{quote} \mid \text{syntax} \mid \mathbf{VAR}(id) \mid val$
 $\Sigma ::= \text{binding store, } name \rightarrow (\overline{scp} \rightarrow name)$
 $name ::= \text{a token such as x, egg, or lambda}$
 $scp ::= \text{a token that represents a scope}$
 $ph ::= integer$
 $\widehat{scp} ::= scp \mid \bullet$
 $\hat{\Sigma} ::= \langle \Sigma, \overline{scp}, \overline{scp} \rangle$

eval : $ph\ ast\ scp\ \xi\ \hat{\Sigma} \rightarrow \langle val, \hat{\Sigma} \rangle$

$eval_{ph}[\mathbf{APP}(\mathbf{lvalue}, ast_{id}), scp_i, \xi, \hat{\Sigma}] = \langle \xi(\text{resolve}_{ph}[id_{result}, \Sigma_2]), \hat{\Sigma}_2 \rangle$
 subject to $eval_{ph}[ast_{id}, scp_i, \xi, \hat{\Sigma}] = \langle id_{result}, \hat{\Sigma}_2 \rangle, \hat{\Sigma}_2 = \langle \Sigma_2, _, _ \rangle$

$eval_{ph}[\mathbf{APP}(\mathbf{lexpand}, ast, ast_{stops}), scp_i, \xi, \hat{\Sigma}] = \langle \text{flip}_{ph}[stx_{exp}, scp_i], \hat{\Sigma}_4 \rangle$
 subject to $eval_{ph}[ast, scp_i, \xi, \hat{\Sigma}] = \langle stx, \hat{\Sigma}_2 \rangle,$

$eval_{ph}[ast_{stops}, scp_i, \xi, \hat{\Sigma}_2] = \langle \mathbf{LIST}(id_{stop}, \dots), \hat{\Sigma}_3 \rangle,$
 $\{var \rightarrow \text{unstop}[\xi(var)] \mid var \in \text{dom}(\xi)\} = \xi_{unstops}, \hat{\Sigma}_3 = \langle \Sigma_3, _, _ \rangle,$
 $\text{resolve}_{ph}[id_{stop}, \Sigma_3], \dots = name_{stop}, \dots,$
 $\xi_{unstops} + \{name_{stop} \rightarrow \text{STOP}(\xi_{unstops}(name_{stop}))\} \dots = \xi_{stops},$
 $\text{expand}_{ph}[\text{flip}_{ph}[stx, scp_i], \xi_{stops}, \hat{\Sigma}_3] = \langle stx_{exp}, \hat{\Sigma}_4 \rangle$

$eval_{ph}[\mathbf{APP}(\mathbf{lbinder}, ast_{id}), scp_i, \xi, \hat{\Sigma}] = \langle \text{prune}_{ph}[id_{result}, \overline{scp}_{u2}], \hat{\Sigma}_2 \rangle$
 subject to $eval_{ph}[ast_{id}, scp_i, \xi, \hat{\Sigma}] = \langle id_{result}, \hat{\Sigma}_2 \rangle, \hat{\Sigma}_2 = \langle _, _, \overline{scp}_{u2} \rangle$

$eval_{ph}[\mathbf{APP}(ast_{fun}, ast_{arg}), scp, \xi, \hat{\Sigma}] = eval_{ph}[ast_{body}[var \leftarrow val_{arg}], scp, \xi, \hat{\Sigma}_3]$
 subject to $eval_{ph}[ast_{fun}, scp, \xi, \hat{\Sigma}] = \langle \mathbf{FUN}(var, ast_{body}), \hat{\Sigma}_2 \rangle,$
 $eval_{ph}[ast_{arg}, scp, \xi, \hat{\Sigma}_2] = \langle val_{arg}, \hat{\Sigma}_3 \rangle$

$eval_{ph}[\mathbf{APP}(prim, ast_{arg}, \dots), scp, \xi, \hat{\Sigma}] = \langle \delta(prim, val_{arg}, \dots), \hat{\Sigma}_2 \rangle$
 subject to $eval^*[ph, (), (ast_{arg} \dots), scp, \xi, \hat{\Sigma}] = \langle (val_{arg} \dots), \hat{\Sigma}_2 \rangle$

$eval_{ph}[val, scp, \xi, \hat{\Sigma}] = \langle val, \hat{\Sigma} \rangle$

unstop : $transform \rightarrow transform$

$\text{unstop}[\text{STOP}(transform)] = transform$
 $\text{unstop}[transform] = transform$

$\delta(\mathbf{stx-e}, \mathbf{STX}(val, ctx)) = val$
 $\delta(\mathbf{mk-stx}, atom, \mathbf{STX}(val, ctx)) = \mathbf{STX}(atom, ctx)$
 $\delta(\mathbf{mk-stx}, \mathbf{LIST}(stx, \dots), \mathbf{STX}(val, ctx)) = \mathbf{STX}(\mathbf{LIST}(stx, \dots), ctx)$

parse : $ph\ stx\ \Sigma \rightarrow ast$

$\text{parse}_{ph}[\mathbf{STX}(\mathbf{LIST}(id_{lambda}, id_{arg}, stx_{body}), ctx), \Sigma] = \mathbf{FUN}(\mathbf{VAR}(\text{resolve}_{ph}[id_{arg}, \Sigma]), \text{parse}_{ph}[stx_{body}, \Sigma])$
 subject to $\text{resolve}_{ph}[id_{lambda}, \Sigma] = \text{lambda}$

$\text{parse}_{ph}[\mathbf{STX}(\mathbf{LIST}(id_{quote}, stx), ctx), \Sigma] = \text{strip}[stx]$
 subject to $\text{resolve}_{ph}[id_{quote}, \Sigma] = \text{quote}$

$\text{parse}_{ph}[\mathbf{STX}(\mathbf{LIST}(id_{syntax}, stx), ctx), \Sigma] = stx$
 subject to $\text{resolve}_{ph}[id_{syntax}, \Sigma] = \text{syntax}$

$\text{parse}_{ph}[\mathbf{STX}(\mathbf{LIST}(stx_{rator}, stx_{rand}, \dots), ctx), \Sigma] = \mathbf{APP}(\text{parse}_{ph}[stx_{rator}, \Sigma], \text{parse}_{ph}[stx_{rand}, \Sigma], \dots)$

$\text{parse}_{ph}[id, \Sigma] = \mathbf{VAR}(\text{resolve}_{ph}[id, \Sigma])$

resolve : $ph \text{ id } \Sigma \rightarrow name$

resolve_{ph}[[**STX**('name', ctx), Σ]] = name_{biggest}
 subject to $\Sigma(name) = \{\overline{scp}_{bind} \leftarrow name_{bind}, \dots\}$,
 biggest-subset[[ctx(ph), $\{\overline{scp}_{bind}, \dots\}$]] = $\overline{scp}_{biggest}$,
 $\{\overline{scp}_{bind} \leftarrow name_{bind}, \dots\}(\overline{scp}_{biggest}) = name_{biggest}$

resolve_{ph}[[**STX**('name', ctx), Σ]] = name

biggest-subset : $\overline{scp} \{ \overline{scp}, \dots \} \rightarrow \overline{scp}$

biggest-subset[[$\overline{scp}_{ref}$, $\{\overline{scp}_{bind}, \dots\}$]] = $\overline{scp}_{biggest}$
 subject to $\overline{scp}_{biggest} \subseteq \overline{scp}_{ref}$, $\overline{scp}_{biggest} \in \{\overline{scp}_{bind}, \dots\}$,
 $\overline{scp}_{bind} \subseteq \overline{scp}_{ref} \Rightarrow \overline{scp}_{bind} \subseteq \overline{scp}_{biggest}$

strip : $stx \rightarrow val$

strip[[**STX**(atom, ctx)]] = atom
 strip[[**STX**(**LIST**(stx, ...), ctx)]] = **LIST**(strip[[stx]], ...)

$$\begin{aligned}
& \text{expand} : ph \text{ stx } \xi \hat{\Sigma} \rightarrow \langle stx, \hat{\Sigma} \rangle \\
& \text{expand}_{ph} \llbracket \mathbf{STX}(\mathbf{LIST}(id, stx, \dots), ctx), \xi, \hat{\Sigma} \rrbracket = \langle \mathbf{STX}(\mathbf{LIST}(id, stx, \dots), ctx), \hat{\Sigma} \rangle \\
& \text{subject to } \hat{\Sigma} = \langle \Sigma, _ , _ \rangle, \xi(\text{resolve}_{ph} \llbracket id, \Sigma \rrbracket) = \text{STOP}(_) \\
& \text{expand}_{ph} \llbracket \mathbf{STX}(\mathbf{LIST}(id_{lam}, id_{arg}, stx_{bdy}), ctx), \xi, \langle \Sigma, \overline{scp}_p, \overline{scp}_u \rangle \rrbracket \\
& = \langle \mathbf{STX}(\mathbf{LIST}(id_{lam}, id_{new}, stx_{bdy2}), ctx), \langle \Sigma_4, \overline{scp}_p, \overline{scp}_u \rangle \rangle \\
& \text{subject to } \text{resolve}_{ph} \llbracket id_{lam}, \Sigma \rrbracket = \text{lambda}, \text{alloc-name} \llbracket \Sigma \rrbracket = \langle name_{new}, \Sigma_1 \rangle, \\
& \quad \text{alloc-scope} \llbracket \Sigma_1 \rrbracket = \langle scp_{new}, \Sigma_2 \rangle, \text{add}_{ph} \llbracket id_{arg}, scp_{new} \rrbracket = id_{new}, \\
& \quad \Sigma_2 + \{id_{new} \rightarrow name_{new}\} = \Sigma_3, \xi + \{name_{new} \rightarrow \text{VAR}(id_{new})\} = \xi_{new}, \\
& \quad \langle \Sigma_3, \{scp_{new}\} \cup \overline{scp}_p, \emptyset \rangle = \hat{\Sigma}_3, \\
& \quad \text{expand}_{ph} \llbracket \text{add}_{ph} \llbracket stx_{bdy}, scp_{new} \rrbracket, \xi_{new}, \hat{\Sigma}_3 \rrbracket = \langle stx_{bdy2}, \langle \Sigma_4, _ , _ \rangle \rangle \\
& \text{expand}_{ph} \llbracket \mathbf{STX}(\mathbf{LIST}(id_{quote}, stx), ctx), \xi, \hat{\Sigma} \rrbracket = \langle \mathbf{STX}(\mathbf{LIST}(id_{quote}, stx), ctx), \hat{\Sigma} \rangle \\
& \text{subject to } \hat{\Sigma} = \langle \Sigma, _ , _ \rangle, \text{resolve}_{ph} \llbracket id_{quote}, \Sigma \rrbracket = \text{quote} \\
& \text{expand}_{ph} \llbracket \mathbf{STX}(\mathbf{LIST}(id_{syntax}, stx), ctx), \xi, \hat{\Sigma} \rrbracket \\
& = \langle \mathbf{STX}(\mathbf{LIST}(id_{syntax}, stx_{pruned}), ctx), \hat{\Sigma} \rangle \\
& \text{subject to } \hat{\Sigma} = \langle \Sigma, \overline{scp}_p, \overline{scp}_u \rangle, \text{resolve}_{ph} \llbracket id_{syntax}, \Sigma \rrbracket = \text{syntax}, \text{prune}_{ph} \llbracket stx, \overline{scp}_p \rrbracket = stx_{pruned} \\
& \text{expand}_{ph} \llbracket \mathbf{STX}(\mathbf{LIST}(id_{ls}, id, stx_{rhs}, stx_{body}), ctx), \xi, \langle \Sigma, \overline{scp}_p, \overline{scp}_u \rangle \rrbracket = \langle stx_{result}, \langle \Sigma_6, \overline{scp}_p, \overline{scp}_u \rangle \rangle \\
& \text{subject to } \text{resolve}_{ph} \llbracket id_{ls}, \Sigma \rrbracket = \text{let-syntax}, \text{alloc-name} \llbracket \Sigma \rrbracket = \langle name_{new}, \Sigma_1 \rangle, \\
& \quad \text{alloc-scope} \llbracket \Sigma_1 \rrbracket = \langle scp_{new}, \Sigma_2 \rangle, \text{add}_{ph} \llbracket id, scp_{new} \rrbracket = id_{new}, \\
& \quad \Sigma_2 + \{id_{new} \rightarrow name_{new}\} = \Sigma_3, \\
& \quad \text{expand}_{ph+1} \llbracket stx_{rhs}, \xi_{primitives}, \langle \Sigma_3, \emptyset, \emptyset \rangle \rrbracket = \langle stx_{exp}, \langle \Sigma_4, _ , _ \rangle \rangle, \\
& \quad \text{eval}_{ph} \llbracket \text{parse}_{ph+1} \llbracket stx_{exp}, \Sigma_4 \rrbracket, \bullet, \xi, \langle \Sigma_4, \overline{scp}_p, \emptyset \rangle \rrbracket = \langle val_{exp}, \langle \Sigma_5, _ , _ \rangle \rangle, \\
& \quad \xi + \{name_{new} \rightarrow val_{exp}\} = \xi_{new}, \text{add}_{ph} \llbracket stx_{body}, scp_{new} \rrbracket = stx_{body2}, \\
& \quad \text{expand}_{ph} \llbracket stx_{body2}, \xi_{new}, \langle \Sigma_5, \{scp_{new}\} \cup \overline{scp}_p, \emptyset \rangle \rrbracket = \langle stx_{result}, \langle \Sigma_6, _ , _ \rangle \rangle \\
& \text{expand}_{ph} \llbracket stx_{macapp}, \xi, \langle \Sigma, \overline{scp}_p, \overline{scp}_u \rangle \rrbracket = \langle stx_{result}, \hat{\Sigma}_5 \rangle \\
& \text{subject to } stx_{macapp} = \mathbf{STX}(\mathbf{LIST}(id_{mac}, stx_{arg}, \dots), ctx), \xi(\text{resolve}_{ph} \llbracket id_{mac}, \Sigma \rrbracket) = \text{val}, \\
& \quad \text{alloc-scope} \llbracket \Sigma \rrbracket = \langle scp_u, \Sigma_2 \rangle, \text{alloc-scope} \llbracket \Sigma_2 \rrbracket = \langle scp_i, \Sigma_3 \rangle, \\
& \quad \langle \Sigma_3, \{scp_u\} \cup \overline{scp}_p, \{scp_u\} \cup \overline{scp}_u \rangle = \hat{\Sigma}_3, \\
& \quad \text{flip}_{ph} \llbracket \text{add}_{ph} \llbracket stx_{macapp}, scp_u \rrbracket, scp_i \rrbracket = stx_{macapp2}, \\
& \quad \text{eval}_{ph} \llbracket \mathbf{APP}(val, stx_{macapp2}), scp_i, \xi, \hat{\Sigma}_3 \rrbracket = \langle stx_{exp}, \hat{\Sigma}_4 \rangle, \\
& \quad \text{expand}_{ph} \llbracket \text{flip}_{ph} \llbracket stx_{exp}, scp_i \rrbracket, \xi, \hat{\Sigma}_4 \rrbracket = \langle stx_{result}, \hat{\Sigma}_5 \rangle \\
& \text{expand}_{ph} \llbracket \mathbf{STX}(\mathbf{LIST}(stx_{rtor}, stx_{rnd}, \dots), ctx), \xi, \langle \Sigma, \overline{scp}_p, \overline{scp}_u \rangle \rrbracket \\
& = \langle \mathbf{STX}(\mathbf{LIST}(stx_{exp\text{rtor}}, stx_{exp\text{rnd}}, \dots), ctx), \langle \Sigma_1, \overline{scp}_p, \overline{scp}_u \rangle \rangle \\
& \text{subject to } \text{expand}^*_{ph} \llbracket () , (stx_{rtor} \ stx_{rnd} \ \dots), \xi, \langle \Sigma, \overline{scp}_p, \emptyset \rangle \rrbracket = \langle (stx_{exp\text{rtor}} \ stx_{exp\text{rnd}} \ \dots), \Sigma_1 \rangle \\
& \text{expand}_{ph} \llbracket id, \xi, \hat{\Sigma} \rrbracket = \langle id_{new}, \hat{\Sigma} \rangle \\
& \text{subject to } \hat{\Sigma} = \langle \Sigma, _ , _ \rangle, \xi(\text{resolve}_{ph} \llbracket id, \Sigma \rrbracket) = \text{VAR}(id_{new})
\end{aligned}$$

expand* : $ph\ (stx\ \dots)\ (stx\ \dots) \xrightarrow{\Sigma} \langle (stx\ \dots), \Sigma \rangle$

$expand^*_{ph}[(stx_{done}\ \dots), ()], \xi, \langle \Sigma, _, _ \rangle \mathbb{I} = \langle (stx_{done}\ \dots), \Sigma \rangle$

$expand^*_{ph}[(stx_{done}\ \dots), (stx_0\ stx_1\ \dots)], \xi, \langle \Sigma, \overline{scp}_p, \emptyset \rangle \mathbb{I}$
 $= expand^*_{ph}[(stx_{done}\ \dots\ stx_{done0}), (stx_1\ \dots)], \xi, \langle \Sigma_2, \overline{scp}_p, \emptyset \rangle \mathbb{I}$
 subject to $expand_{ph}[(stx_0, \xi, \langle \Sigma, \overline{scp}_p, \emptyset \rangle) \mathbb{I} = \langle stx_{done0}, \langle \Sigma_2, _, _ \rangle \rangle$

prune : $ph\ stx\ \overline{scp} \rightarrow stx$

$prune_{ph}[\mathbf{STX}(atom, ctx), \overline{scp}_p] \mathbb{I} = \mathbf{STX}(atom, ctx + \{ph \rightarrow ctx(ph) \setminus \overline{scp}_p\})$

$prune_{ph}[\mathbf{STX}(\mathbf{LIST}(stx, \dots), ctx), \overline{scp}_p] \mathbb{I} = \mathbf{STX}(\mathbf{LIST}(stx_{pruned}, \dots), ctx + \{ph \rightarrow ctx(ph) \setminus \overline{scp}_p\})$
 subject to $prune_{ph}[(stx, \overline{scp}_p), \dots] = stx_{pruned}, \dots$

add : $ph\ stx\ scp \rightarrow stx$

$add_{ph}[\mathbf{STX}(atom, ctx), scp] \mathbb{I} = \mathbf{STX}(atom, ctx + \{ph \rightarrow \{scp\} \cup ctx(ph)\})$

$add_{ph}[\mathbf{STX}(\mathbf{LIST}(stx, \dots), ctx), scp] \mathbb{I} = \mathbf{STX}(\mathbf{LIST}(add_{ph}[(stx, scp)], \dots), ctx + \{ph \rightarrow \{scp\} \cup ctx(ph)\})$

flip : $ph\ stx\ scp \rightarrow stx$

$flip_{ph}[\mathbf{STX}(atom, ctx), scp] \mathbb{I} = \mathbf{STX}(atom, ctx + \{ph \rightarrow scp \oplus ctx(ph)\})$

$flip_{ph}[\mathbf{STX}(\mathbf{LIST}(stx, \dots), ctx), scp] \mathbb{I} = \mathbf{STX}(\mathbf{LIST}(flip_{ph}[(stx, scp)], \dots), ctx + \{ph \rightarrow scp \oplus ctx(ph)\})$